

Monopoly Deal Cards Game – Phase 2 最终项目说明

- [Monopoly Deal Cards Game – Phase 2 最终项目说明](#)

- [1. 项目概述](#)
- [2. 需求理解与实现范围](#)
 - [2.1 用户需求](#)
 - [2.2 功能需求](#)
 - [2.3 非功能需求](#)
- [3. 系统架构与模块设计](#)
 - [3.1 总体架构](#)
 - [3.2 UML 设计](#)
 - [3.3 关键流程拆分](#)
 - [3.4 核心领域职责与关系](#)
- [4. Clean Design 说明](#)
 - [4.1 类与模块职责清晰](#)
 - [4.2 接口使用](#)
 - [4.3 数据结构选择](#)
 - [4.4 继承关系](#)
- [5. 设计模式应用](#)
- [6. 功能实现说明](#)
 - [6.1 游戏模式](#)
 - [6.2 回合流程](#)
 - [6.3 卡牌与规则](#)
 - [6.4 胜负判断](#)
 - [6.5 存档、读档和加密](#)
 - [6.6 GUI 与用户体验](#)
 - [6.7 多人同步与隐私保护](#)
- [7. 软件工程实践](#)
 - [7.1 需求到实现的追踪](#)
 - [7.2 文档维护](#)
 - [7.3 Git 协作](#)
- [8. 测试与验证](#)
 - [8.1 测试命令](#)
 - [8.2 当前验证结果](#)
 - [8.3 测试覆盖范围](#)
- [9. 运行方式与 Demo 准备](#)
 - [9.1 环境要求](#)
 - [9.2 运行后端服务器](#)
 - [9.3 运行 JavaFX 桌面客户端](#)

- [9.4 运行测试](#)
- [9.5 可展示的 Demo 流程](#)
- [10. 成员分工与贡献](#)
- [11. 对评分标准的对应总结](#)
- [12. 结论](#)

Monopoly Deal Cards Game – Phase 2 最终项目说明

课程项目： Software Engineering Project – Monopoly Deal Cards Game

小组： Group 7

提交阶段： Phase-2 Final Submission

截止日期： 2026 年 6 月 9 日

GitHub Repository: https://github.com/NEVER-AGAIN-RAY/Monopoly_Group7

Brightspace 提交说明：根据课程要求，Phase-2 提交时应上传本 PDF 文档，并在文档中提供 GitHub 仓库链接；代码通过 GitHub 仓库提交，不在 Brightspace 直接上传代码压缩包。

1. 项目概述

本项目实现了一个基于 **Monopoly Deal Cards Game** 的数字卡牌游戏。项目目标不是只完成一个可运行的小游戏，而是按照软件工程课程要求，完整经历需求分析、系统建模、面向对象设计、设计模式应用、模块化实现、GUI 展示、版本控制协作和测试验证等过程。

系统使用 **Java 17** 作为主要开发语言，后端采用 Maven 管理依赖和测试；桌面端 GUI 使用 **JavaFX + FXML**，服务端与客户端之间通过 **WebSocket + JSON** 进行状态同步。项目同时保留了一个 Vite/Vue Web 前端作为辅助展示与调试入口，但课程主要 GUI 交付以 JavaFX 客户端为核心。

与课程要求对应，本项目重点完成了以下内容：

- 正确识别并实现 Monopoly Deal 的核心用户需求和系统需求。
 - 使用清晰的类、接口和模块划分表达游戏组件。
 - 采用 MVC、Singleton、Facade、Observer、Factory Method、Strategy、Memento 等至少五种设计模式。
 - 实现 2-5 人游戏、玩家对战、人与 AI 对战、AI 难度、出牌规则、租金结算、行动牌效果、暂停、存档、读档和胜负判定。
 - 提供 JavaFX GUI，使玩家可以通过图形界面进行摸牌、出牌、查看桌面状态和操作游戏。
 - 使用 Git 进行分布式版本控制，并通过测试验证主要功能。
-

2. 需求理解与实现范围

2.1 用户需求

从玩家角度看，本项目需要提供一个可理解、可操作、能自动执行规则的 Monopoly Deal 数字版本。用户需求可以具体化为以下玩家目标：

具体玩家目标	本项目实现方式
玩家希望快速开始一局 2-5 人 Monopoly Deal	开局界面和 <code>StartSessionRequest</code> 支持人数、模式、AI 难度和先手设置。
玩家希望知道自己现在能做什么，避免违反规则	JavaFX 界面展示回合、手牌、桌面资产和操作按钮；服务端用 <code>TurnPhase</code> 和协议校验阻止非法动作。
玩家希望通过摸牌、部署房产和存银行逐步建立优势	<code>TurnFlowService</code> 支持摸牌、存银行、部署房产、行动牌执行和三次出牌限制。
玩家希望围绕完整房产套组制定策略并最终获胜	<code>PropertySetCalculator</code> 计算完整套组；系统在达到三套完整房产时触发胜利判断。
玩家希望租金、讨债和生日礼金等支付规则由系统自动处理	<code>RentCalculator</code> 和 <code>PaymentSettlement</code> 自动计算金额、选择支付来源并执行多付不找零。
玩家希望行动牌产生真实对抗，例如偷牌、换牌、反制和保护资产	<code>ActionEffectDispatcher</code> 与具体 <code>ActionEffect</code> 实现 Deal Breaker、Sly Deal、Forced Deal、Just Say No 等效果。
玩家希望多人模式中只能看到公开信息，不能看到别人手牌	<code>STATE_UPDATE</code> 只广播公开状态， <code>MY_HAND</code> 只发给对应玩家连接。
玩家希望中断后可以恢复游戏	<code>GameSessionMemento</code> 、 <code>SaveLoadService</code> 和 <code>SaveEncryption</code> 支持保存、读取、自动保存和可选加密。

2.2 功能需求

本项目将课程需求拆分为启动初始化、回合流程、核心玩法、状态反馈、胜负判断和存档暂停六个功能域。实现时优先保证 Monopoly Deal 的核心玩法闭环，再实现多人、AI、存档等增强功能。

功能域	关键实现
游戏启动与初始化	<code>StartSessionRequest</code> 描述开局参数； <code>GameController</code> 初始化玩家、牌堆、回合和快照； <code>MonopolyDealCardFactory</code> 生成标准可游戏牌堆。
回合流程	<code>TurnFlowService</code> 管理 <code>DRAW -> PLAY -> WAITING_FOR_RESPONSE -> END_TURN</code> 状态机；控制摸牌、出牌次数、弃牌和结束回合。
卡牌与行动	<code>Card</code> 抽象出通用卡牌； <code>PropertyCard</code> 、 <code>PropertyWildCard</code> 、 <code>MoneyCard</code> 、 <code>ActionCard</code> 分别处理房产、万能房产、现金和行动牌。
租金与支付	<code>RentCalculator</code> 计算租金； <code>PaymentSettlement</code> 执行银行区/财产区支付，禁止从手牌直接支付，并执行多付不找零规则。
行动牌效果	<code>ActionEffectDispatcher</code> 将行动牌效果码映射到具体效果类，如 <code>Rent</code> 、 <code>Deal Breaker</code> 、 <code>Sly Deal</code> 、 <code>Forced Deal</code> 、 <code>Debt Collector</code> 、 <code>Birthday</code> 、 <code>House</code> 、 <code>Hotel</code> 、 <code>Pass Go</code> 、 <code>Double Rent</code> 、 <code>Just Say No</code> 等。
暂停与存档	<code>PauseVoteService</code> 处理暂停和多人确认； <code>SaveLoadService</code> 处理手动保存、读取和自动保存； <code>GameSessionMemento</code> 捕获完整对局状态。
网络与同步	<code>GameServer</code> 、 <code>MessageDispatcher</code> 、 <code>SessionRegistry</code> 、 <code>MonopolyWebSocketEndpoint</code> 共同处理 <code>WebSocket</code> 连接、消息路由和多人会话隔离。

2.3 非功能需求

非功能需求	对应设计
可用性	GUI 使用卡牌图片、颜色标签、中文/英文资源文件、规则面板和操作提示，降低玩家理解成本。
性能	回合操作、租金计算、状态广播均在内存模型中完成；提供 <code>PerformanceSmokeTest</code> 进行性能烟测。

非功能需求	对应设计
可靠性	非法操作通过协议错误码和异常边界拦截，不让客户端输入直接破坏领域状态。
数据完整性	存档使用 Memento 捕获牌堆、玩家手牌、银行、财产、回合、响应窗口和效果栈，支持恢复完整状态。
隐私	多人模式下公开快照只暴露手牌数量，手牌详情只发给本人连接。
可维护性	按 controller、model、network、dto、persistence、pattern、fx 等包划分职责，并用接口隔离变化。

3. 系统架构与模块设计

3.1 总体架构

项目采用分层架构：

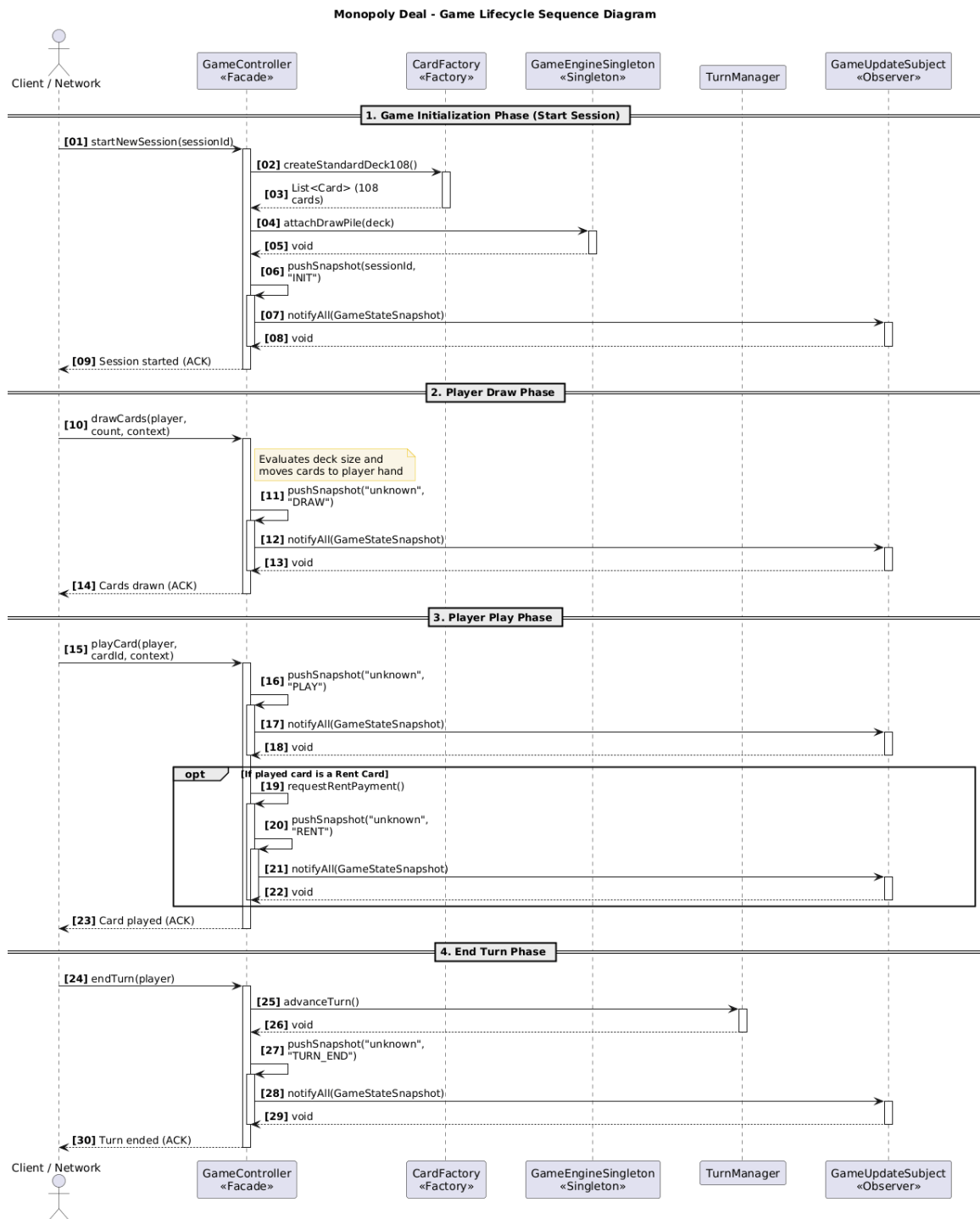
- **View / GUI 层：** JavaFX + FXML，主要包括 `MonopolyFxApp`、`MainController`、`MainView.fxml`、`CardView` 和 `PlayerBoardPanel`。
- **Controller / Facade 层：** `GameController` 作为统一入口，向 GUI、网络层和测试暴露稳定接口。
- **Service 层：** `TurnFlowService`、`EffectStackOrchestrator`、`RentSettlementService`、`AiTurnService`、`PauseVoteService`、`SaveLoadService` 分别负责不同业务流程。
- **Domain Model 层：** `model.card`、`model.player`、`model.effects`、`model.settlement`、`model.core` 保存纯游戏规则和领域状态。
- **Network / Protocol 层：** `GameServer`、`MessageDispatcher`、`SessionRegistry` 和 WebSocket endpoint 负责 JSON 协议与连接管理。
- **Persistence 层：** `GameSessionMemento`、`PersistedCard`、`SaveEncryption` 负责存档和恢复。

这种设计避免将所有逻辑集中在一个大类中，使游戏规则、网络协议、GUI 展示和存档机制可以分别测试和演进。

3.2 UML 设计

项目使用类图、时序图和用例图说明核心结构与主要交互。类图用于展示模型、控制器、网络、持久化和设计模式之间的关系；时序图用于说明从客户端请求到服务端处理再到状态广播的生命周期；用例图用于表达玩家、AI 和系统之间的主要交互。





Sequence Diagram

流程	主要步骤	说明
玩家回合流程	当前玩家摸牌 -> 最多三次出牌 -> 可能进入响应窗口 -> 必要时弃牌 -> 结束回合	重点展示 <code>TurnPhase</code> 状态机和非法操作拦截。
租金与反制流程	玩家打出租金牌 -> 计算金额 -> 目标玩家选择 Just Say No 或支付 -> 系统结算资产转移	重点展示核心 Monopoly Deal 玩法和效果栈。
存档读档流程	玩家请求保存 -> Memento 捕获全局状态 -> 可选加密 -> 读取时恢复牌堆、玩家分区、回合和响应状态	重点展示 Memento、数据完整性和恢复能力。

3.4 核心领域职责与关系

为了使类图职责更加清楚，本项目将核心对象关系概括如下：

类/模块	核心职责	关系说明
<code>GameController</code>	对外 Facade，协调各服务并推送快照	组合 <code>TurnManager</code> 、 <code>TurnFlowService</code> 、 <code>SaveLoadService</code> 等服务。
<code>TurnFlowService</code>	维护回合状态机和出牌规则	依赖玩家、卡牌、行动效果和胜利判断。
<code>EffectStackOrchestrator</code>	管理租金/行动牌响应窗口和 Just Say No 链	与 <code>StackResponseState</code> 、 <code>EffectStackEntry</code> 、 <code>PaymentSettlement</code> 协作。
<code>Player</code>	保存玩家手牌、银行、房产和行动区	<code>HumanPlayer</code> 和 <code>AIPlayer</code> 继承它；AI 额外组合策略对象。
<code>Card</code>	所有卡牌的抽象父类	<code>PropertyCard</code> 、 <code>MoneyCard</code> 、 <code>ActionCard</code> 等继承它。
<code>ActionEffect</code>	行动牌效果接口	每种行动牌效果是一个独立实现类，降低条件分支复杂度。
<code>RentCalculator</code>	根据颜色、套组和建筑计算租金	与 <code>PropertySetCalculator</code> 、 <code>RentTierTable</code> 协作。
<code>GameServer</code>	接收 WebSocket 消息并广播状态	实现 <code>GameUpdateObserver</code> ，监听 controller 的状态变化。

类/模块	核心职责	关系说明
GameSessionMemento	捕获和恢复完整对局状态	支持保存、读档和自动保存。

4. Clean Design 说明

课程评分中 Clean Design 占 20%。本项目主要从类职责、接口设计、数据结构、继承关系和设计模式五个方面满足要求。

4.1 类与模块职责清晰

项目共有 100 多个 Java 主源码文件，核心逻辑按包分层：

包	职责
controller	对局生命周期、回合流程、AI 回合、效果栈、暂停、存档、租金结算。
model.card	卡牌抽象与具体卡牌类型。
model.player	玩家抽象、人类玩家、AI 玩家和玩家分区。
model.effects	行动牌效果接口、上下文、执行结果和具体效果实现。
model.settlement	房产套组、租金计算、支付结算和偷牌规则。
dto	WebSocket JSON 载荷和快照对象。
network	WebSocket 接入、连接管理、消息解析与广播。
persistence	对局快照、卡牌持久化镜像和存档加密。
pattern	课程要求的典型设计模式实现。
fx	JavaFX 桌面客户端。

原本容易膨胀的 GameController 被设计为 Facade，复杂流程下沉到单一职责服务中。例如，回合状态机在 TurnFlowService，响应窗口和 Just Say No 反制链在 EffectStackOrchestrator，存档逻辑在 SaveLoadService，AI 回合在 AiTurnService。

4.2 接口使用

课程要求“对于行为相似但不继承同一父类的类，应定义并使用接口”。本项目使用了多处接口来隔离行为：

接口	作用
Playable	表示可以被打出的对象，提供可玩性校验。
Payable	表示可用于支付的对象，统一获取支付面值。
ActionEffect	所有行动牌效果的统一执行接口。
AiPlayStrategy	不同 AI 难度和策略的统一决策接口。
GameUpdateSubject / GameUpdateObserver	状态更新的发布与订阅接口。
ClientConnection	屏蔽具体 WebSocket Session，实现连接抽象。
AiGameBridge	AI 复用人类玩家同一套出牌校验入口。

这些接口使系统在增加新卡牌效果、新 AI 策略、新连接类型或新展示端时，不需要重写核心业务规则。

4.3 数据结构选择

本项目根据游戏语义选择数据结构：

- 使用 `List<Card>` 表示手牌、牌堆、弃牌堆、银行区和行动区，因为这些区域需要保持顺序并支持按索引操作。
- 使用玩家列表管理回合顺序，`TurnManager` 通过当前下标推进回合。
- 使用 Map/双向索引管理 session、connection 和 playerId 的对应关系，保证多人连接能准确路由。
- 使用 enum 表示回合阶段、卡牌类型、建筑等级、响应角色等有限状态，减少字符串错误。
- 使用效果栈对象表示租金、Double Rent、Just Say No 等链式响应，避免把复杂响应逻辑写成散乱条件判断。
- 使用 DTO 对象定义 JSON 消息结构，避免在业务逻辑中直接操作未校验的原始 JSON。

4.4 继承关系

卡牌和玩家是本项目最自然的继承场景：

- `Card` 是抽象基类，`PropertyCard`、`PropertyWildCard`、`ActionCard`、`MoneyCard` 表示不同牌型。
- `Player` 是抽象玩家，`HumanPlayer` 和 `AIPlayer` 分别表示人类玩家和电脑玩家。

继承只用于表达“is-a”关系；对于可支付、可打出、可观察、可决策等跨层行为，则优先使用接口和组合，避免继承层次过深。

5. 设计模式应用

课程明确要求至少使用五种设计模式。本项目实际使用并能够在代码中定位的设计模式如下：

设计模式	代码位置	作用
MVC	model、controller、 fx/MainController、 MainView.fxml	将游戏规则、控制流程和 GUI 展示分离。
Singleton	GameEngineSingleton	管理抽牌堆、弃牌堆、洗牌和牌数统计，保证同一局内核心牌堆状态一致。
Facade	GameController	向 GUI、网络层和测试提供统一入口，隐藏内部服务协作细节。
Observer	GameUpdateSubject、 GameUpdateObserver、 DefaultGameUpdateSubject	当游戏状态变化时通知网络层广播 STATE_UPDATE。
Factory Method	CardFactory、 MonopolyDealCardFactory	统一创建标准 Monopoly Deal 牌堆，保证牌型和数量一致。
Strategy	AiPlayStrategy、 EasyAiPlayStrategy、 NormalAiPlayStrategy、 HardAiPlayStrategy、 SearchLookaheadAiPlayStrategy	将 AI 决策算法从玩家实体中拆出，便于替换不同难度。
Strategy	ActionEffect 及各具体效果类	将不同行动牌效果封装成独立策略，避免巨大 switch 或 if-else。
Memento	GameSessionMemento、 SessionPlayerMemento、 PersistedCard	捕获和恢复完整游戏状态，实现存档与读档。

这些模式不是为了“凑数量”而加入，而是服务于实际复杂度：牌堆创建需要 Factory，状态广播需要 Observer，AI 难度需要 Strategy，存档需要 Memento，网络 and GUI 入口需要 Facade，整局牌堆状态需要 Singleton，整体界面与规则分离需要 MVC。

6. 功能实现说明

6.1 游戏模式

系统支持 2–5 名玩家。开局请求由 `StartSessionRequest` 描述，包含 `sessionId`、玩家数量、游戏模式、AI 难度和是否随机先手。HVM 模式中，电脑玩家由 `AIPlayer` 组合不同 `AiPlayStrategy`；PVP 模式中，多个客户端通过 `WebSocket` 加入同一会话。

6.2 回合流程

每个玩家回合包括：

1. 摸牌阶段：普通情况摸 2 张，空手时摸 5 张。
2. 出牌阶段：最多执行 3 次出牌动作，包括存银行、部署房产、执行行动牌。
3. 响应阶段：租金、讨债、偷牌等行动可以进入等待响应窗口，允许 Just Say No 或放弃响应。
4. 弃牌与结束回合：手牌超过 7 张时必须弃牌，之后进入下一名玩家回合。

`TurnFlowService` 显式维护 `TurnPhase`，因此非法时机操作会被拦截，不会直接修改游戏状态。

6.3 卡牌与规则

项目实现了 Monopoly Deal 的主要卡牌类别：

- 房产牌和万能房产牌。
- 现金牌。
- 租金牌、双色租金牌和任意色租金牌。
- Pass Go。
- Double The Rent。
- Debt Collector。
- It's My Birthday。
- Sly Deal。
- Forced Deal。
- Deal Breaker。
- House 和 Hotel。
- Just Say No / Rent Waiver。

行动牌由 `ActionEffectDispatcher` 统一分发到具体效果类。租金计算由 `RentCalculator` 和 `RentTierTable` 处理，支付结算由 `PaymentSettlement` 保证“不从手牌支付”和“多付不找零”。

6.4 胜负判断

系统持续监控玩家财产区。当玩家达到三套完整房产时，系统生成胜利状态并结束对局。房产套组判断由 `PropertySetCalculator` 统一完成，避免 GUI、AI 和后端各自重复判断。

6.5 存档、读档和加密

存档功能通过 `GameSessionMemento` 捕获完整会话状态，包括：

- 牌堆和弃牌堆。
- 所有玩家的手牌、银行区、财产区和行动区。
- 当前玩家与回合阶段。
- 效果栈和响应窗口。
- 租金/支付相关状态。

`SaveEncryption` 支持可选 AES-GCM 加密。未设置密钥时可以使用明文 JSON 便于调试；设置 `monopoly.saveKey` 后可以保护本地存档不被轻易篡改。

6.6 GUI 与用户体验

JavaFX 客户端满足课程“Consider having GUI for the game”的要求，并进一步提供：

- FXML 声明式布局。
- 卡牌图片资源。
- 玩家桌面面板。
- 手牌渲染。
- 按钮操作和目标选择对话框。
- 游戏规则说明。
- 中文/英文资源文件。
- 与 WebSocket 后端实时同步。

GUI 不直接执行业务规则，而是通过 JSON 消息请求后端处理；后端返回最新状态，客户端刷新界面。这保证了规则权威性集中在服务端，减少多人对战中的状态不一致。

6.7 多人同步与隐私保护

WebSocket 协议使用统一 JSON envelope。常见消息包括：

- `START_SESSION`
- `PLAY_CARD`
- `END_TURN`
- `SAVE_GAME`
- `LOAD_GAME`
- `STATE_UPDATE`
- `MY_HAND`
- `ERROR`
- `PING` / `PONG`

其中 `STATE_UPDATE` 是公共状态广播，只包含每名玩家的公开信息；`MY_HAND` 是私有手牌消息，只发送给对应玩家。这样可以防止一名玩家从普通广播中看到其他玩家手牌。

7. 软件工程实践

7.1 需求到实现的追踪

项目维护了 `docs/requirements/requirements.md` 作为需求基线，并维护 `docs/implementation/requirement-trace-and-deviations.md` 记录实现状态、偏差和后续动作。这体现了软件工程中的需求追踪意识，而不是只在最后交代码。

7.2 文档维护

项目文档包括：

文档	作用
<code>README.md</code>	项目简介、运行命令、时间计划和文档导航。
<code>docs/requirements/requirements.md</code>	功能与非功能需求基线。
<code>docs/architecture/uml_source.md</code>	架构说明、模块职责、UML 源码和导出图。
<code>docs/interface/websocket-protocol.md</code>	WebSocket 消息类型、载荷结构和协议约定。
<code>docs/implementation/requirement-trace-and-deviations.md</code>	需求覆盖、偏差和风险记录。
<code>docs/ENGINEERING.md</code>	工程变更记录和协作约定。

这些文档使项目在 demo 和答辩时可以清楚解释“为什么这样设计”和“如何验证功能”。

7.3 Git 协作

项目使用 GitHub 作为分布式版本仓库。提交信息使用 `feat`、`fix`、`test`、`docs`、`chore` 等语义化前缀，便于识别每次修改的目的。例如：

- `feat(fx): expand start page game guide`
- `feat(fx): polish hand area UX and lobby info nav`
- `chore: remove research/ML scaffolding not required for submission`

Git 历史保留了需求、架构、功能、GUI、测试和文档的增量演进过程，符合课程对分布式版本控制和自解释 commit message 的要求。

8. 测试与验证

课程测试要求是实现多个有意义的测试用例，覆盖主要功能，不需要测试 `getter` 和 `setter`。本项目使用 JUnit 5，并通过 Maven 统一运行测试。

8.1 测试命令

```
mvn -q test
```

8.2 当前验证结果

在 2026 年 6 月 1 日执行 `mvn -q test` :

指标	结果
测试用例总数	259
Failures	0
Errors	0
Skipped	0

8.3 测试覆盖范围

项目共有 50 个测试类，覆盖以下方面：

测试领域	示例
牌堆与规则	DeckCompositionTest 、 DeckShuffleOrderTest 、 MonopolyDealBankValuesTest
回合流程	OverflowDiscardRuleTest 、 DoubleRentNextRentTest 、 DeckBoundaryRuleTest
行动牌效果	DealBreakerEffectTest 、 ForcedDealEffectTest 、 StealCardEffectTest 、 HouseHotelEffectTest
租金与支付	RentCalculatorPartialSetTest 、 RentCalculatorBuildingBonusTest 、 PaymentSettlementExplicitTest
存档与加密	GameSessionMementoTest 、 SaveEncryptionTest 、 SaveLoadIntegrationTest
网络协议	PingProtocolTest 、 PlayProtocolValidationTest 、

测试领域	示例
	GameServerEndToEndSmokeTest
多人模式	GameServerMultiSessionIsolationTest 、 GameServerPvpSaveVoteTest 、 GameServerPvpLoadVoteTest
AI 策略	AiStrategyProfileDifferentiationTest 、 SearchLookaheadAiPlayStrategyTest
性能烟测	PerformanceSmokeTest

测试重点放在核心业务方法、状态流转、规则边界和协议契约上，而不是简单 getter/setter，因此更符合课程要求。

9. 运行方式与 Demo 准备

9.1 环境要求

- JDK 17 或以上。
- Maven 3.9 或以上。
- 可选：Node.js，用于构建 Web/Vite 前端。

9.2 运行后端服务器

```
mvn -q exec:java
```

默认 WebSocket endpoint:

```
ws://localhost:8025/ws
```

9.3 运行 JavaFX 桌面客户端

```
mvn javafx:run
```

9.4 运行测试

```
mvn -q test
```

9.5 可展示的 Demo 流程

建议 demo 时按以下顺序展示：

- 1. 启动 WebSocket server。
- 2. 启动 JavaFX 客户端。
- 3. 创建 2–5 人对局，选择 HVM 或 PVP。
- 4. 展示摸牌、部署房产、存银行和行动牌。
- 5. 展示租金计算和支付规则。
- 6. 展示 Just Say No 响应窗口。
- 7. 展示胜利条件。
- 8. 展示保存与读取。
- 9. 展示测试命令和测试结果。
- 10. 结合 UML 图解释核心设计模式。

10. 成员分工与贡献

根据课程要求，每次提交都应包含成员贡献表，并明确说明是否存在贡献明显高于其他成员的情况。本组按照五人小组进行分工，整体贡献保持均衡。

成员	主要职责	具体贡献	贡献比例
Liu Zongrun	架构设计与核心逻辑	参与总体架构设计、类结构设计、核心控制流程、代码重构和模块职责划分。	20%
Lei Yurun	UML、GUI 与项目组织	负责 UML 图整理、界面设计讨论、JavaFX 展示优化、项目文档组织和 demo 准备。	20%
Wang Tingdong	需求整理与规则校验	负责用户需求和系统需求整理、Monopoly Deal 规则核对、需求文档与实现对应检查。	20%
Qiu Siqu	测试与质量检查	负责功能测试、规则边界验证、UML 与代码一致性检查、运行兼容性检查。	20%
Liu Yuhao	协作、文档与审查	协助需求文档、提交材料、展示内容和最终报	20%

成员	主要职责	具体贡献	贡献比例
		告审查，并参与多人流程验证。	

说明：本项目代码实现、需求整理、UML、测试、GUI 和最终提交材料之间存在不同类型的工作量。我们在表中将贡献按课程项目全过程计算，而不仅仅按单一 Git commit 数量计算。若最终提交前团队确认某位成员贡献明显高于其他成员，应按照老师要求在此表中更新比例并明确说明。

11. 对评分标准的对应总结

老师评分项	占比	本项目对应证据
Game requirements and Modelling	20%	需求文档、UML 类图/时序图/用例图、项目完成计划和分工表。
Clean design	20%	分层架构、单一职责服务、接口隔离、合理继承、正确数据结构、至少七类设计模式。
Functionality	40%	支持核心游戏规则、GUI、HVM/PVP、AI、租金、行动牌、响应链、存档读档、多人同步和胜利判断。
Teamwork	10%	GitHub 仓库、语义化 commit、五人任务分工、贡献表和协作文档。
Testing	10%	50 个 JUnit 测试类、259 个测试用例、0 failures、0 errors，覆盖规则、网络、存档、AI 和性能烟测。

12. 结论

本项目严格围绕课程要求完成了 Monopoly Deal Cards Game 的需求分析、UML 建模、面向对象设计、Java 实现、GUI 展示、多人通信、存档机制、设计模式应用和测试验证。项目不仅关注“功能能运行”，也关注软件工程课程强调的 clean design、接口抽象、模块划分、可维护性、版本控制和可测试性。

最终提交时，本 PDF 文档将作为 Phase-2 说明材料上传到 Brightspace，并在文档中提供 GitHub 仓库链接。项目源代码、提交历史、测试和文档均保存在 GitHub 仓库中，便于老师和 TA 检查、运行和答辩提问。